



**Università
di Brescia**

From Observation to Symbolic Logic:

Learning PDDL Action Models from Observed Plans and Transitions

Sara Moglia, Luca Puggnetti

Artificial Intelligence

Outline

- 1 Introduction
- 2 Framework Positioning
- 3 Agent Characteristics
- 4 Agent Implementation
- 5 Experiments and Results

Introduction

The Problem: Manual Domain Specification

The classic approach to planning

- An expert *manually* defines preconditions and effects of every action.
- The planner generates a plan starting from the provided domain.

Practical limitations

- It is a costly, fragile operation prone to omissions.
- It depends on perfect a priori knowledge of the domain.
- It is inapplicable in initially unknown environments.

Our solution: the inverted flow

In this work, the agent *reconstructs* preconditions and effects by observing the execution of its own actions over time, without the symbolic domain being known a priori.

The Scenario: Stochastic Grid World

Domain elements

- A 4×4 grid, a robot, an object.
- Three available actions: move, pick, drop.

Environment challenges

- **Actuator noise:** actions can fail (slip, pick_fail, drop_misplace).
- **Partial observability:** sensors can hide some state predicates with a certain probability (mask_prob).

The agent's goal

Transition from concrete and specific observations (e.g., *"robot at (2, 3) executes pick"*) to abstract, parametric, and reusable logical operators: `pick(?robot, ?obj, ?x, ?y)`.

Framework Positioning

Comparison with reference approaches

	OLAM [1]	OffLAMPT [3]	PAL [2]	This work
Modality	Online	Offline	Online	Online
Obs. Type	Symbolic	Symbolic	Continuous ($\mathbb{R}^n$)	Symbolic
Observability	Full	Partial	Partial	Partial
Output	PDDL Op.	PDDL Op.	Extens. Mod.	PDDL Op.
Guarantees	Formal	Formal	Empirical	Empirical

What we inherit:

- From **OLAM**: online learning, updates based only on successes, and generation of parametric operators via the lifting operation.
- From **OffLAMPT**: the inertia principle to handle partially observed states.

Key differences from the state of the art

Compared to OLAM

OLAM requires full state observability and conducts goal-directed exploration by computing what it still does not know about the current domain, whereas our system is designed for partially observable environments and conducts a hybrid exploration consisting of a manual part and a random part.

Compared to OffLAMPT

OffLAMPT operates offline on historical traces to derive minimal logical models with formal guarantees of soundness and completeness, while our system operates online using statistical thresholds to tolerate actuator noise.

Compared to PAL

PAL tackles the perception problem by building a mapping to translate continuous sensory data into discrete states, while our system operates on already symbolic logical predicates; consequently, the proposed framework bypasses the problem of building an abstract representation of the environment.

Agent Characteristics

From concrete state to symbolic predicates

The simulator translates the physical state into a set of logical predicates. For example: `at(robot, (x,y)), at(box, (x,y)), holding(robot, box), handempty`.

Direct consequence on learning

Working with logical sets, the effects of an action become simple set differences between the arrival state (s') and the starting state (s):

$$\text{ADD} = s' \setminus s \quad \text{DEL} = s \setminus s'$$

Execution example (pick)

- **Initial state (s):** `at(robot, (2,3)), at(box, (2,3)), handempty`
- **Final state (s'):** `at(robot, (2,3)), at(box, held), holding(robot, box)`
- **ADD effects:** `at(box, held), holding(robot, box)`
- **DEL effects:** `at(box, (2,3)), handempty`

Partial Observability and Noise

The partial blindness of sensors (mask_prob)

- Each true predicate in the state (initial or final) is removed from the observation with independent probability mask_prob .
- This introduces a **fundamental ambiguity**: if a predicate is missing from the observation, the agent does not know if it became false or if it is simply hidden.

Actuator noise

- The robot can slip (slip), fail to grasp (pick_fail) or release the object in the wrong cell (drop_misplace). Respectively with probabilities: slip_prob , pick_fail_prob , $\text{drop_misplace_prob}$.
- **Handling**: Failed transitions are recorded in the global history, but are *excluded* from the calculation of the action's nominal effects. The model learns only from successes.

The Output: Pseudo-PDDL Operators

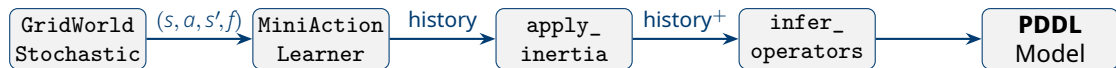
At the end of the exploration, the agent outputs a logical model for each action. Specific constants (e.g., coordinates) are replaced by abstract variables (lifting).

```
(:action pick
:parameters (?robot ?obj ?x1 ?y1)
:precondition
  (and (at ?robot ?x1 ?y1)
        (at ?obj ?x1 ?y1)
        (handempty ?robot))
:add-effects
  (holding ?robot ?obj)
  (at ?obj held)
:del-effects
  (at ?obj ?x1 ?y1)
  (handempty ?robot))
```

The presence of preconditions and effects in this block is dictated exclusively by exceeding the statistical thresholds τ_{pre} and τ_{eff} .

Agent Implementation

The Implementation Pipeline



The four sequential phases

- 1 **Generation:** The simulator produces transitions by interacting with the environment (manual plans to ensure coverage + random exploration for variety).
- 2 **Statistical collection:** The learner counts the frequency of predicates.
- 3 **Data recovery:** Inertia estimates and reinserts predicates hidden by sensors.
- 4 **Lifting:** Data is parameterized and converted into PDDL syntax.

Pseudocode: Inference Pipeline

Algorithm 1 Main domain inference pipeline

Initialize environment `GridWorldStochastic` and `Learner`

Phase 1: Manual plans to ensure examples of `pick` and `drop`

for all manual plan π **do**

 Execute π , observe states s, s' (with masking) and update learner

end for

Phase 2: Random exploration

for $ep \leftarrow 1$ **to** number of episodes **do**

 Choose applicable action, execute, observe masked states

 Update learner with $(s, a, s', \text{failed})$

end for

Calculate empirical probabilities of effects

Apply **inertia completion** to history

Infer **lifted** operators (with PDDL variables) via statistical thresholds

return inferred operators

Pseudocode: Statistical Update

Algorithm 2 Learner update on a transition

Require: State s , action a , next state s' , flag failed

Save $(s, a, s', \text{failed})$ in the global history

if failed is true **then**

Increment failures and stop

return

end if

Increment success count for operator a

Add s to the precondition examples

for every atom $p \in s' \setminus s$ **do**

Increment **positive effects** count (ADD)

end for

for every atom $p \in s \setminus s'$ **do**

Increment **negative effects** count (DEL)

end for

How the Learner learns: statistical calculation

For each action a , the `MiniActionModelLearner` calculates empirical probabilities by dividing occurrences by the total number of successful executions $N(a_s)$. $T_s(a)$ represents the set of successful transitions for action a , while $\text{pre_ex}(a)$ is the set of relative initial states before execution.

Preconditions

A predicate p is considered a precondition if it is frequently present before the action is successfully executed:

$$p \in \text{Pre}(a) \iff \frac{|\{s \in \text{pre_ex}(a) : p \in s\}|}{N(a_s)} \geq \tau_{\text{pre}}$$

Effects (Positive and Negative)

They are deduced if their variation between the initial and final state exceeds the confidence threshold τ_{eff} . We symmetrically consider both additions (ADD) and deletions (DEL):

$$p \in \text{ADD}(a) \iff \frac{|\{(s, s') \in T_s(a) : p \in (s' \setminus s)\}|}{N(a_s)} \geq \tau_{\text{eff}}$$

$$p \in \text{DEL}(a) \iff \frac{|\{(s, s') \in T_s(a) : p \in (s \setminus s')\}|}{N(a_s)} \geq \tau_{\text{eff}}$$

Information recovery: The inertia principle

The partial observability problem

Due to `mask_prob`, the learner receives punctured states. This artificially lowers the calculated frequencies, preventing the τ thresholds from being crossed.

Application of inertia (`apply_inertia`)

If a predicate p does not fall within the possible effects altered by action a , it is assumed that its truth value must remain constant (inertia).

$$p \in s, \text{ sig}(p) \notin \text{sig}(\text{EFF}(a)) \Rightarrow p \in s'$$

The predicate signature (`sig`)

The `sig` function abstracts from concrete coordinates to understand the type of fact. For example, `sig(at(robot,1,1)) = "at:robot"`. If the action moves the robot, inertia understands that position is mutable data for that action, and does *not* propagate the old coordinate.

Two validation levels of the learned model

In the framework, we evaluate the quality of the final result according to two distinct criteria.

1. Weak criterion (`check_pddl_inference`)

- Verifies the *structural completeness* of the operator.
- Simply checks that the four fundamental fields (parameters, preconditions, add-effects, del-effects) are not empty.
- It is used in narrative experiments (iterative scenarios).

2. Strong criterion (`is_strictly_valid`)

- Verifies *semantic correctness* using the Unified Planning Framework (UPF).
- Requires the operator to respect the logical rules of the domain (e.g., in `move`, the starting coordinates must disappear and new ones must appear).
- It is used for robustness evaluations (Heatmaps).

Experiments and Results

Convergence in the four narrative scenarios

We tested the learning speed (measured based on the weak structural criterion) by progressively adding exploration blocks.

Scenario	Noise	Masking	Move	Pick/Drop	Manual Exec.
Ideal Baseline	0%	0%	✓	✓	20
High env. noise	40%	0%	✓	✓	30
Low visibility	0%	40%	✓	✓	70
Low Exploration	0%	0%	✓	✓	20

Observations: Actuator noise negligibly slows down learning, as failures are discarded. The real difficulty is posed by low visibility (masking), which requires many more attempts (70) before inertia can stabilize the data.

Final Dashboard (Robustness Heatmaps)

For an exhaustive test, we calculated the percentage of *semantically valid* models by simultaneously varying noise and masking from 0.0 to 0.5. The shown run converges at **1600 episodes**; other executions maintain the same qualitative trend.

How to read the heatmaps

- **Columns:** from left to right, MOVE, PICK, DROP.
- **Rows:** each row is a different noise scenario.
- **Cells:** percentage of semantically valid operators according to `is_strictly_valid`.

Heatmaps: single noise scenarios

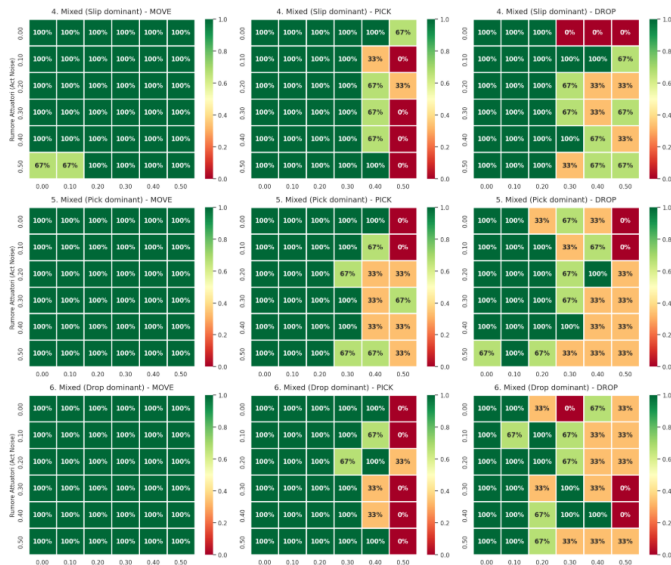


Displayed rows

- 1 Only slip_prob varies: locomotion problem.
- 2 Only pick_fail_prob varies: picking problem.
- 3 Only drop_misplace_prob varies: release/positioning problem.

It can be seen that `move` remains almost always stable, while `pick` and especially `drop` begin to lose reliability when the specific action disturbance becomes high.

Heatmaps: asymmetric noise scenarios



Displayed rows

- 4 Noise dominated by `slip_prob`: weight $5/3$, the other two at $2/3$.
- 5 Noise dominated by `pick_fail_prob`: weight $5/3$, the other two at $2/3$.
- 6 Noise dominated by `drop_misplace_prob`: weight $5/3$, the other two at $2/3$.

When one noise dominates the others, the cells worsen especially in the column of the corresponding action: the observed behavior is thus consistent with the nature of the introduced disturbance.

Heatmaps: symmetric noise scenario



Displayed row

- 7 Symmetric noise: `slip_prob`, `pick_fail_prob` and `drop_misplace_prob` contribute equally.

In the symmetric case, the degradation is more distributed: `move` remains relatively robust, while `pick` and `drop` are more affected by the combination of actuator noise and masking.

Pattern analysis from the Dashboard

Move Action — Extremely robust

- It almost always reaches 100% validity, resisting very well to both noise and masking. It has a simple structure and inertia intervenes effectively.

Pick Action — Highly sensitive to visibility

- The degradation is sharp when masking exceeds the 0.3 threshold.
- The action involves complex interaction between robot and object: it requires co-observing many predicates simultaneously. If even one is missing, the operator fails the semantic check.

Drop Action — Sensitive to positioning

- It suffers particularly from the increase in specific failure probability (`drop_misplace`). The final spatial effect becomes unstable, making it difficult to extract the correct logical signature.

Conclusions

Framework strengths

- Fully inspectable pipeline, combining statistical learning and backward logical inference.
- Remarkable resilience for basic navigation actions.
- Inertial completion recovers sufficient information to mitigate sensory masking up to 30%.

Encountered limitations

- Lack of formal or theoretical convergence guarantees.
- Strong dependence on the manual balancing of statistical thresholds (τ).

References I



L. Lamanna et al.

Online Learning of Action Models for PDDL Planning.

IJCAI-21, pp. 4112–4118, 2021.



L. Lamanna et al.

On-line Learning of Planning Domains from Sensor Data in PAL.

AAAI-21, pp. 11862–11869, 2021.



L. Lamanna et al.

Lifted Action Models Learning from Partial Traces.

Artificial Intelligence, 339:104256, 2025.